

CONTEST CODEBOOK

INSTITUTION / UNIVERSITY

University of Rajshahi

CONTEST TEAM IDENTITY

RU_Parity

STL

STL

```
C++ STL QUICK TEMPLATE
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
using vi=vector<int>;
using vll=vector<ll>;
using pii=pair<int,int>;
using pll=pair<ll,ll>;
const int INF=1e9;
const ll LINF=4e18;
#define fast_io
ios::sync_with_stdio(false);cin.tie(nullptr)

CONTAINERS
vector<int> v;  vector<int> v(n),v2(n,x);
vector<vector<int>> g(n,vector<int>(m,0));
vector<vector<vector<int>>>
dp(n,vector<vector<int>>(m,vector<int>(k,x)));
pair<int,int> p={x,y};
pair<int,pair<int,int>> np={a,{b,c}};  set<int>
s;  multiset<int> ms;  unordered_set<int> us;
map<int,int> mp;
unordered_map<int,int> ump;
map<int,vector<int>> mpv;  set<pair<int,int>>
sp;  stack<int> st;  queue<int> q;
deque<int> dq;  priority_queue<int> mx;
priority_queue<int,vector<int>,greater<int>>
mn;  string str;
int arr[100005],mat[105][105];

VECTOR / ALGORITHMS
sort(v.begin(),v.end());
sort(v.rbegin(),v.rend());
reverse(v.begin(),v.end());
rotate(v.begin(),v.begin()+k,v.end());
rotate(v.begin(),v.end()-k,v.end());
auto mn_it=min_element(v.begin(),v.end());
auto mx_it=max_element(v.begin(),v.end());
ll sum=accumulate(v.begin(),v.end(),0LL);  int
freq=count(v.begin(),v.end(),x);  auto
it=find(v.begin(),v.end(),x);
bool exists=binary_search(v.begin(),v.end(),x);
auto lb=lower_bound(v.begin(),v.end(),x);
auto ub=upper_bound(v.begin(),v.end(),x);
next_permutation(v.begin(),v.end());

VECTOR OPERATIONS
v.push_back(x);  v.pop_back();  v.size();
v.empty();  v.clear();  v.resize(n);
v.assign(n,x);
v.erase(v.begin()+i);  v.insert(v.begin()+i,x);
v.front();  v.back();

SET / MULTISET
s.insert(x);  s.erase(x);  s.erase(s.find(x));
s.count(x);  s.find(x);  s.lower_bound(x);
s.upper_bound(x);
ms.insert(x);  ms.erase(ms.find(x));
ms.erase(x);  ms.count(x);  ms.lower_bound(x);
ms.upper_bound(x);
us.insert(x);  us.erase(x);  us.count(x);
us.find(x);
```

STL (cont.)

```
MAP
mp[x]++;  mp[x]=y;  mp.insert({x,y});  mp.erase(x);
mp.count(x);  mp.find(x);  ump[x]++;  ump.erase(x);
ump.count(x);  ump.find(x);

STACK / QUEUE / DEQUE
st.push(x);  st.pop();  st.top();  q.push(x);  q.pop();
q.front();  q.back();  dq.push_front(x);
dq.push_back(x);
dq.pop_front();  dq.pop_back();  dq.front();  dq.back();

PRIORITY QUEUE
mx.push(x);  mx.pop();  mx.top();  mn.push(x);
mn.pop();  mn.top();

STRING
str.size();  str.empty();  str.substr(pos,len);
str.find(t);  str.find(t)!=string::npos;
reverse(str.begin(),str.end());
sort(str.begin(),str.end());
count(str.begin(),str.end(),ch);
str.erase(pos,len);  str.insert(pos,t);

NUMERIC / BIT
max(a,b);  min(a,b);  max({a,b,c});  min({a,b,c});
gcd(a,b);  lcm(a,b);  __gcd(a,b);
__builtin_popcount(x);
__builtin_popcountll(x);  __builtin_clz(x);
__builtin_ctz(x);  __builtin_clzll(x);
__builtin_ctzll(x);  abs(x);

ITERATION
for(auto x:v){}
for(auto &x:v){}
for(auto &[key,val]:mp){}
for(auto &row:g)for(auto &x:row)x=0;

UNIQUE IN FIRST-APPEARANCE ORDER
vll get_unique_sequential(const vll& a){
    unordered_set<ll> seen; vll res;
    for(ll
x:a)if(seen.insert(x).second)res.push_back(x);
    return res;
}

RANDOM
mt19937
rng(chrono::steady_clock::now().time_since_epoch().count());
int random_value=uniform_int_distribution<int>(l,r)
(rng);

DEBUG
cerr<<x<<endl;

COMMON COMPLEXITIES
vector push_back: amortized O(1), access O(1),
insert/erase middle O(n)
sort: O(n log n), reverse: O(n), accumulate/count/find:
O(n)
binary_search/lower_bound/upper_bound on sorted random-
access range: O(log n)  set/map: insert/find/erase O(log
n)
unordered_set/unordered_map: average O(1), worst O(n)
priority_queue push/pop O(log n), top O(1)
stack/queue/deque end operations O(1)
```

COMPARATOR

Contest Checklist & Edge Cases

NOTE

Pre-Submit Sanity Checklist
Integer Overflow: *Check if $N \cdot M$ or running sum exceeds $2 \cdot 10^9$. Use `1LL << k`, cast `1LL` a * b % MOD`.*
* **Array Bounds:** 1-indexed vs 0-indexed vectors; verify max bounds (e.g. $N \le 2 \cdot 10^5$).
* **Graph Traps:** Disconnected graph, multiple edges between same pair, self-loops, $N = 1$ or empty components.
* **Floating Point:** Use `1e-9` epsilon tolerance; avoid direct `==` with `double`.
* **Interactive Problems:** Always flush output with `cout << endl;` or `cout.flush();`.

comparator

```cpp
//COMPARATOR

#include <bits/stdc++.h>
using namespace std;

// 1. Integer: ASC / DESC
bool ascending(int a, int b) { return a < b; }
bool descending(int a, int b) { return a > b; }

// 2. Pair: First / Second ASC
bool firstAscending(pair<int,int> a,
pair<int,int> b) { return a.first < b.first; }
bool secondAscending(pair<int,int> a,
pair<int,int> b) { return a.second < b.second;
}

// 3. Pair: First ASC, tie → Second ASC / DESC
bool pairCompareAsc(pair<int,int> a,
pair<int,int> b) {
    if (a.first != b.first) return a.first <
b.first;
    return a.second < b.second;
}
bool pairCompareDesc(pair<int,int> a,
pair<int,int> b) {
    if (a.first != b.first) return a.first <
b.first;
    return a.second > b.second;
}

// 4. String: Alphabetical / Shorter first
bool alphabetical(string a, string b) { return
a < b; }
bool shorterFirst(string a, string b) { return
a.size() < b.size(); }

// 5. Interval {start, end}: End ASC / DESC,
tie → Start ASC
bool endAscending(pair<int,int> a,
pair<int,int> b) {
```

comparator (cont.)

```cpp
    if (a.second != b.second) return a.second <
b.second;
    return a.first < b.first;
}
bool endDescending(pair<int,int> a,
pair<int,int> b) {
    if (a.second != b.second) return a.second >
b.second;
    return a.first < b.first;
}

int main() {
    vector<int> v = {4,2,5,1,3};
    sort(v.begin(), v.end(), ascending);
    sort(v.begin(), v.end(), descending);

    vector<pair<int,int>> p = {{2,4},{1,3},
{2,3},{1,8}};
    sort(p.begin(), p.end()); // First ASC, tie
→ Second ASC
    sort(p.begin(), p.end(), pairCompareDesc);

    vector<string> words =
{"pear","a","banana","fig","kiwi"};
    sort(words.begin(), words.end(),
alphabetical);
    sort(words.begin(), words.end(),
shorterFirst);

    vector<pair<int,int>> intervals = {{1,5},
{2,3},{4,7},{2,6}};
    sort(intervals.begin(), intervals.end(),
endAscending);
    sort(intervals.begin(), intervals.end(),
endDescending);
    return 0;
}

/* QUICK REFERENCE
sort(v.begin(), v.end());                    //
ASC
sort(v.begin(), v.end(), greater<int>());  //
DESC
sort(p.begin(), p.end());                    //
Pair: first ASC, second ASC
sort(p.begin(), p.end(), pairCompareDesc); //
First ASC, second DESC
sort(words.begin(), words.end());            //
Alphabetical
*/
```

STRING

string

```cpp
// BUILT-IN STRING
/*
Construction: string s; string s="abc"; string
s(n,c);
Size: s.size(), s.length(), s.empty(),
s.clear();
```

string (cont.)

```
Access: s[i], s.front(), s.back();
Append: s+="abc"; s+='x'; s.append("abc");
s.append(3,'x');
Modify: s.push_back('x'); s.pop_back();
Substring: s.substr(pos,len); s.substr(pos);
Search: s.find("abc"); s.find('a',pos);
s.rfind("abc");
Not found: string::npos
Insert/Erase/Replace: s.insert(pos,"abc");
s.insert(pos,k,'x');
s.erase(pos,len); s.erase(it);
s.replace(pos,len,"abc");
Compare: s==t; s<t; s.compare(t);
Algorithms: reverse(s.begin(),s.end());
sort(s.begin(),s.end());
count(s.begin(),s.end(),'a');
Conversion: stoi(s); stoll(s); to_string(x);
Input: cin>>s; getline(cin,s);
*/

// CUSTOM FUNCTIONS
bool is_palindrome(const string &s) {
    int l = 0, r = (int)s.size() - 1;
    while (l < r) {
        if (s[l] != s[r]) return false;
        l++; r--;
    }
    return true;
}
string reverse_string(string s) {
    reverse(s.begin(),s.end());
    return s;
}
vector<int> freq_lower(const string &s) {
    vector<int> cnt(26,0);
    for (char c : s) cnt[c-'a']++;
    return cnt;
}
bool is_anagram(const string &a, const string
&b) {
    if (a.size() != b.size()) return false;
    return freq_lower(a) == freq_lower(b);
}
bool is_rotation(const string &a, const string
&b) {
    if (a.size() != b.size()) return false;
    return (a+a).find(b) != string::npos;
}
vector<string> split(const string &s, char
delim) {
    vector<string> res;
    string cur;
    for (char c : s) {
        if (c == delim) {
            if (!cur.empty())
res.push_back(cur);
            cur.clear();
        } else cur += c;
    }
    if (!cur.empty()) res.push_back(cur);
    return res;
}
string trim(string s) {
    int l = 0, r = (int)s.size()-1;
```

string (cont.)

```
    while (l <= r && isspace(s[l])) l++;
    while (l <= r && isspace(s[r])) r--;
    return s.substr(l,r-l+1);
}
vector<string> all_substrings(const string &s)
{
    vector<string> res;
    int n = s.size();
    for (int i = 0; i < n; i++)
        for (int len = 1; i+len <= n; len++)
            res.push_back(s.substr(i,len));
    return res;
}

// KMP
vector<int> lps_array(const string &p) {
    int m = p.size();
    vector<int> lps(m,0);
    for (int i = 1, len = 0; i < m; ) {
        if (p[i] == p[len]) lps[i++] = ++len;
        else if (len) len = lps[len-1];
        else lps[i++] = 0;
    }
    return lps;
}
vector<int> kmp_search(const string &s, const
string &p) {
    vector<int> res, lps = lps_array(p);
    for (int i = 0, j = 0; i < (int)s.size(); )
{
        if (s[i] == p[j]) i++, j++;
        if (j == (int)p.size()) {
            res.push_back(i-j);
            j = lps[j-1];
        } else if (i < (int)s.size() && s[i] !=
p[j]) {
            if (j) j = lps[j-1];
            else i++;
        }
    }
    return res;
}

// Z ALGORITHM
vector<int> z_array(const string &s) {
    int n = s.size();
    vector<int> z(n,0);
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i <= r) z[i] = min(r-i+1,z[i-l]);
        while (i+z[i] < n && s[z[i]] ==
s[i+z[i]]) z[i]++;
        if (i+z[i]-1 > r) l = i, r = i+z[i]-1;
    }
    return z;
}
// ROLLING HASH
struct StringHash {
    static const long long mod = 1000000007;
    static const long long base = 91138233;
    vector<long long> h, p;

    StringHash(const string &s) {
        int n = s.size();
        h.assign(n+1,0); p.assign(n+1,1);
```

string (cont.)

```
        for (int i = 0; i < n; i++) {
            h[i+1] = (h[i]*base+s[i])%mod;
            p[i+1] = (p[i]*base)%mod;
        }
    }

    long long get(int l, int r) {
        long long res = (h[r+1]-h[l]*p[r-
l+1])%mod;
        if (res < 0) res += mod;
        return res;
    }
}
```

BINARY SEARCH

binary search

```
// BINARY SEARCH
void binary_search_min_satisfy() {
    auto ok = [&](ll mid) { return true; };
    ll l = 0, r = 1e9, mid, ans = -1;
    while (l <= r) {
        mid = l + (r - l) / 2;
        if (ok(mid)) ans = mid, r = mid - 1;
        else l = mid + 1;
    }
    cout << ans << endl;
}
void binary_search_max_satisfy() {
    auto ok = [&](ll mid) { return true; };
    ll l = 1, r = 1e9, mid, ans = -1;
    while (l <= r) {
        mid = l + (r - l) / 2;
        if (ok(mid)) ans = mid, l = mid + 1;
        else r = mid - 1;
    }
    cout << ans << endl;
}
void binary_search_array_satisfy(vll &a, ll
target) {
    ll l = 0, r = (ll)a.size() - 1, mid;
    bool found = false;
    while (l <= r) {
        mid = l + (r - l) / 2;
        if (a[mid] == target) { found = true;
break; }
        else if (a[mid] < target) l = mid + 1;
        else r = mid - 1;
    }
    if (found) cout << "Target found at index "
<< mid << endl;
    else cout << "Target not found" << endl;
}
```

DIFFERENCE ARRAY

DIFF ARRAY 1D

```
// DIFFERENCE ARRAY
const int MAXN = 2e5 + 5;
ll diff1D[MAXN], a1D[MAXN];

void update1D(int l, int r, ll val) {
    diff1D[l] += val; diff1D[r + 1] -= val;
}

void build1D(int n) {
    for (int i = 1; i <= n; i++) {
        diff1D[i] += diff1D[i - 1]; a1D[i] =
diff1D[i];
    }
}

void clear1D(int n) {
    for (int i = 0; i <= n + 2; i++) diff1D[i]
= 0, a1D[i] = 0;
}
void solve() {
    int n = 5; clear1D(n);
    update1D(2, 4, 5); update1D(1, 3, 2);
    build1D(n);
    // Output: 2 7 7 5 0
}
```

BIT MANUPULATION

Bit manupulation basic

```
#include <bits/stdc++.h>
using namespace std;

// 1. Built-in Functions
// __builtin_popcount(x),
__builtin_popcountll(x): set bit count
// __builtin_clz(x), __builtin_clzll(x):
leading zeros
// __builtin_ctz(x), __builtin_ctzll(x):
trailing zeros
// __builtin_parity(x), __builtin_parityll(x):
odd/even set bit count
// __lg(x): highest set bit index, x > 0

// 2. Basic Bit Tricks (0-based index i)
bool checkBit(long long n, int i) { return n &
(1LL << i); }
long long setBit(long long n, int i) { return n
| (1LL << i); }
long long clearBit(long long n, int i) { return
n & ~(1LL << i); }
long long toggleBit(long long n, int i) {
return n ^ (1LL << i); }
long long lowestSetBit(long long n) { return n
& -n; }
long long removeLowestSetBit(long long n) {
return n & (n - 1); }
```

Bit manupulation basic (cont.)

```
bool isPowerOfTwo(long long n) { return n > 0
&& !(n & (n - 1)); }
// Caution: For signed long long, keep i in [0,
62].

// 3. Bit Utilities
int highestSetBit(unsigned long long n) {
    return n ? 63 - __builtin_clzll(n) : -1;
}
int lowestSetBitIndex(unsigned long long n) {
    return n ? __builtin_ctzll(n) : -1;
}
int countBits(unsigned long long n) { return
__builtin_popcountll(n); }
int countBitsSlow(unsigned long long n) {
    int c = 0;
    while (n) n &= n - 1, c++;
    return c;
}
// Caution: clz/ctz built-ins on zero are
undefined.

// 4. XOR of 0 to n
long long pref_xor(long long n) {
    if (n % 4 == 0) return n;
    if (n % 4 == 1) return 1;
    if (n % 4 == 2) return n + 1;
    return 0;
}
// XOR from l to r: pref_xor(r) ^ pref_xor(l -
1)

// 5. Bitmask
// Full mask for n bits: (1LL << n) - 1
// Check bit: mask & (1LL << i)
// All masks: for (int mask = 0; mask < (1 <<
n); mask++)

vector<int> getSetBits(unsigned long long mask)
{
    vector<int> v;
    for (int i = 0; i < 64; i++)
        if (mask & (1ULL << i)) v.push_back(i);
    return v;
}
// Caution: 1LL << n requires a valid shift;
for signed long long, n ⩽ 62 is safe.

// 6. All Non-empty Submasks
vector<long long> getSubmasks(long long mask) {
    vector<long long> v;
    for (long long sub = mask; sub; sub = (sub
- 1) & mask)
        v.push_back(sub);
    return v;
}
// Includes no zero submask. Add 0 separately
if needed.

// 7. Gray Code
vector<int> grayCode(int n) {
    vector<int> g;
    for (int i = 0; i < (1 << n); i++)
g.push_back(i ^ (i >> 1));
    return g;
}
// Caution: 1 << n and int storage limit the
usable n.

// 8. SOS DP: Sum over Subsets
// f[mask] = sum of original f[sub] for every
sub ⊆ mask
void SOSdp(vector<int> &f, int n) {
    for (int i = 0; i < n; i++)
        for (int mask = 0; mask < (1 << n);
mask++)
            if (mask & (1 << i)) f[mask] +=
f[mask ^ (1 << i)];
}
// Caution: f must have at least (1 << n)
elements; run once on original values.

// 9. TSP Bitmask DP: Minimum Hamiltonian Cycle
from 0
int tsp(int n, vector<vector<int>> &cost) {
    if (n == 1) return cost[0][0];
    int N = 1 << n, INF = 1e9;
    vector<vector<int>> dp(N, vector<int>(n,
INF));
    dp[1][0] = 0;
    for (int mask = 1; mask < N; mask++) {
        for (int last = 0; last < n; last++) {
            if (!(mask & (1 << last)) ||
dp[mask][last] == INF) continue;
            for (int nxt = 0; nxt < n; nxt++) {
                if (mask & (1 << nxt))
continue;
                int nm = mask | (1 << nxt);
                dp[nm][nxt] = min(dp[nm][nxt],
dp[mask][last] + cost[last][nxt]);
            }
        }
    }
    int ans = INF;
    for (int i = 1; i < n; i++)
        if (dp[N - 1][i] != INF) ans = min(ans,
dp[N - 1][i] + cost[i][0]);
    return ans;
}
// Caution: O(n^2 * 2^n) time and O(n * 2^n)
memory; use only for small n.
```

bitmsk

```
#include <bits/stdc++.h>
using namespace std;
#define fast_io ios::sync_with_stdio(false);
cin.tie(nullptr)
#define ll long long

// 1. Print Mask: LSB → MSB
void bitmask(int n) {
    for (int mask = 0; mask < (1 << n); mask++)
{
```

bitmsk (cont.)

```
        cout << mask << " → ";
        for (int k = 0; k < n; k++) cout <<
((mask >> k) & 1) << ' ';
        cout << '\n';
    }
}

// 2. Basic Bit Operations (0-based index k)
bool isSet(int mask, int k) { return mask & (1
<< k); }
int setBit(int mask, int k) { return mask | (1
<< k); }
int clearBit(int mask, int k) { return mask & ~
(1 << k); }
int toggleBit(int mask, int k) { return mask ^
(1 << k); }
int countBits(int mask) { return
__builtin_popcount(mask); }
int lowestBit(int mask) { return mask & -mask;
}
int removeLowestBit(int mask) { return mask &
(mask - 1); }
// Caution: int mask-এর জন্য k সাধারণত [0, 30]
রাখো।

// 3. Iterate All Masks: O(2^n)
void iterateMasks(int n) {
    for (int mask = 0; mask < (1 << n); mask++)
{
        // Process mask
    }
}

// 4. Iterate Elements in a Mask
void iterateElements(int mask, int n) {
    for (int i = 0; i < n; i++)
        if (mask & (1 << i)) {
            // Process element i
        }
}

// 5. Iterate All Non-empty Submasks
void iterateSubmasks(int mask) {
    for (int sub = mask; sub; sub = (sub - 1) &
mask) {
        // Process sub
    }
}
// Include empty submask separately if needed:
sub = 0.

// 6. Iterate Set Bits Only
void iterateSetBits(int mask) {
    while (mask) {
        int bit =
__builtin_ctz((unsigned)mask);
        // Process bit index
        mask &= (mask - 1);
    }
}
// Caution: __builtin_ctz(0) is undefined.

// 7. Generate All Subsets
```

bitmsk (cont.)

```
vector<vector<int>>
generateSubsets(vector<int>& a) {
    int n = a.size();
    vector<vector<int>> res;
    for (int mask = 0; mask < (1 << n); mask++)
{
        vector<int> cur;
        for (int i = 0; i < n; i++)
            if (mask & (1 << i))
cur.push_back(a[i]);
        res.push_back(cur);
    }
    return res;
}
// Caution: O(n * 2^n) time and memory.

// 8. Subset Sum for a Given Mask
ll subsetSum(vector<int>& a, int mask) {
    ll sum = 0;
    for (int i = 0; i < (int)a.size(); i++)
        if (mask & (1 << i)) sum += a[i];
    return sum;
}

// 9. Bitmask DP: Add One Unselected Element
void bitmaskDP(vector<int>& a) {
    int n = a.size();
    vector<ll> dp(1 << n, 0);
    for (int mask = 0; mask < (1 << n); mask++)
        for (int i = 0; i < n; i++)
            if (!(mask & (1 << i))) {
                int nmask = mask | (1 << i);
                dp[nmask] = max(dp[nmask],
dp[mask] + a[i]);
            }
}
// Caution: This transition maximizes selected-
element sum only when the
// intended DP matches this recurrence. For
general problems, define dp state first.

// 10. Print Mask: MSB → LSB
void printMask(int mask, int n) {
    for (int i = n - 1; i >= 0; i--) cout <<
((mask >> i) & 1);
    cout << '\n';
}
// Caution: 1 << n requires a safe shift; use
small n for int bitmask DP.
```

MODULAR ARITHMETIC

MOD

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll MOD = 1e9 + 7;

// 1. Modular Arithmetic
```

MOD (cont.)

```
// Addition:       (a + b) % m
// Subtraction:    (a - b + m) % m
// Multiplication: (a * b) % m
// Normalize negative: (x % m + m) % m
// Division:       a * modInverse(b, m) % m
// Inverse exists iff gcd(b, m) = 1.

// 2. Binary Exponentiation: (a^b) % m
ll modPow(ll a, ll b, ll m = MOD) {
    ll res = 1;
    a = (a % m + m) % m;
    while (b > 0) {
        if (b & 1) res = res * a % m;
        a = a * a % m;
        b >>= 1;
    }
    return res;
}
// Caution: m > 0, b ≥ 0. Multiplication must not overflow ll.

// 3. Modular Inverse: Prime MOD (Fermat's Little Theorem)
// a^(-1) % m = a^(m-2) % m, if m is prime and a % m ≠ 0.
ll modInv(ll a, ll m = MOD) {
    return modPow(a, m - 2, m);
}
// Caution: Requires prime m and gcd(a, m) = 1.

// 4. Modular Inverse: Any Modulus (Extended Euclid)
ll extendedGCD(ll a, ll b, ll &x, ll &y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1, g = extendedGCD(b, a % b, x1, y1);
    x = y1; y = x1 - (a / b) * y1;
    return g;
}
ll modInvGeneral(ll a, ll m) {
    ll x, y, g = extendedGCD(a, m, x, y);
    if (g != 1) return -1; // Inverse does not exist
    return (x % m + m) % m;
}

// 5. Modular Arithmetic Examples
// ll add = (a % m + b % m) % m;
// ll sub = ((a % m - b % m) + m) % m;
// ll mul = (a % m) * (b % m) % m;
// ll power = modPow(a, b, m);
// ll div = (a % m) * modInv(b, m) % m;

// 6. Useful Properties
// (a + b) % m = ((a % m) + (b % m)) % m
// (a - b) % m = ((a % m) - (b % m) + m) % m
// (a * b) % m = ((a % m) * (b % m)) % m
// (a^b) % m = modPow(a, b, m)
```

COMBINATRICS

combinatrics

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll MOD = 1e9 + 7;
const int MAXN = 1e6;
vector<ll> fact(MAXN + 1), invfact(MAXN + 1);

// 1. Binary Exponentiation: (a^b) % MOD
ll binpow(ll a, ll b) {
    ll res = 1;
    a %= MOD;
    while (b > 0) {
        if (b & 1) res = res * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return res;
}

// 2. Modular Inverse: a^(-1) % MOD
ll modinv(ll a) { return binpow(a, MOD - 2); }
// Caution: MOD must be prime and a % MOD ≠ 0.

// 3. Precompute Factorial and Inverse Factorial
void init_fact() {
    fact[0] = 1;
    for (int i = 1; i ≤ MAXN; i++) fact[i] = fact[i - 1] * i % MOD;
    invfact[MAXN] = modinv(fact[MAXN]);
    for (int i = MAXN - 1; i ≥ 0; i--)
        invfact[i] = invfact[i + 1] * (i + 1) % MOD;
}
// Call init_fact() once before using nCr() or nPr().
// Caution: 0 ≤ n ≤ MAXN; this factorial-inverse method requires n < MOD.

// 4. Combination: nCr = n! / (r! * (n-r)!)
ll nCr(ll n, ll r) {
    if (r < 0 || r > n || n > MAXN) return 0;
    return fact[n] * invfact[r] % MOD * invfact[n - r] % MOD;
}

// 5. Permutation: nPr = n! / (n-r)!
ll nPr(ll n, ll r) {
    if (r < 0 || r > n || n > MAXN) return 0;
    return fact[n] * invfact[n - r] % MOD;
}

// 6. Simple Factorial (without MOD)
ll factorial(int n) {
    ll res = 1;
    for (int i = 1; i ≤ n; i++) res *= i;
    return res;
}
// Caution: Overflows quickly. Use only for very small n.
```

combinatrics (cont.)

```
// 7. Simple nCr (without MOD)
ll nCr_simple(ll n, ll r) {
    if (r < 0 || r > n) return 0;
    r = min(r, n - r);
    ll res = 1;
    for (ll i = 1; i <= r; i++) res = res * (n
- r + i) / i;
    return res;
}
// Caution: Intermediate multiplication can
overflow ll.

// 8. Simple nPr (without MOD)
ll nPr_simple(ll n, ll r) {
    if (r < 0 || r > n) return 0;
    ll res = 1;
    for (ll i = 0; i < r; i++) res *= n - i;
    return res;
}
// Caution: Exact result can overflow ll.

// 9. Stars and Bars: k identical items into n
non-empty boxes
// Answer = C(k-1, n-1), when k >= n >= 1.
ll stars_and_bars_nonempty(ll k, ll n) {
    if (n < 1 || k < n) return 0;
    return nCr(k - 1, n - 1);
}

// k identical items into n boxes, empty boxes
allowed
// Answer = C(k+n-1, n-1), when n >= 1.
ll stars_and_bars(ll k, ll n) {
    if (k < 0 || n < 1 || k + n - 1 > MAXN)
return 0;
    return nCr(k + n - 1, n - 1);
}

// 10. Product of Choices: each category i has
(k[i]+1) choices
// Answer = product(k[i]+1) % MOD
ll categoryWays(vector<ll>& k) {
    ll ans = 1;
    for (ll x : k) ans = ans * ((x + 1) % MOD)
% MOD;
    return ans;
}

/* FORMULA QUICK REFERENCE
   Factorial:       n! = 1 * 2 * ... * n
   Permutation:     nPr = n! / (n-r)!
   Combination:     nCr = n! / (r! * (n-r)!)
   Symmetry:        nCr = nC(n-r)
   Relation:        nPr = nCr * r!
   Identities:      nC0 = nCn = 1, nP0 = 1
   Simple nCr:      reduce r = min(r, n-r)
*/
```

NUMBER THEORY

gcd_lcm

```
// GCD function (Euclidean Algorithm)
long long gcd(long long a, long long b) {
    while (b != 0) {
        long long r = a % b;
        a = b;
        b = r;
    }
    return a;    // range GCD → initial value =
0
}

// LCM function
long long lcm(long long a, long long b) {
    return a / gcd(a, b) * b;    // range LCM →
initial value = 1
}
```

NUM

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
using i128 = __int128_t;
using u128 = __uint128_t;
#define fast_io ios::sync_with_stdio(false);
cin.tie(nullptr)

const int N = 2e6 + 5, MAXP = 200005;
vector<int> primes, prime_divisors[MAXP];
int spf[N], mobius[N];
ll phi_arr[N];

// 1. GCD / LCM
ll gcd(ll a, ll b) { return b ? gcd(b, a % b) :
abs(a); }
ll lcm(ll a, ll b) { return a / gcd(a, b) * b;
}
bool coprime(ll a, ll b) { return gcd(a, b) ==
1; }
// Caution: lcm may overflow.

// 2. Extended GCD: a*x + b*y = gcd(a,b)
ll extended_gcd(ll a, ll b, ll &x, ll &y) {
    if (!b) { x = 1; y = 0; return a; }
    ll x1, y1, g = extended_gcd(b, a % b, x1,
y1);
    x = y1; y = x1 - (a / b) * y1;
    return g;
}

// 3. Modular Arithmetic
ll mod_add(ll a, ll b, ll m) { return ((a % m +
b % m) % m + m) % m; }
ll mod_sub(ll a, ll b, ll m) { return ((a % m -
b % m) % m + m) % m; }
ll mod_mul(ll a, ll b, ll m) {
```

NUM (cont.)

```
    return (ll)((i128)((a % m + m) % m) * ((b %
m + m) % m) % m);
}
ll mod_pow(ll a, ll b, ll m) {
    ll r = 1 % m;
    a = (a % m + m) % m;
    while (b > 0) {
        if (b & 1) r = mod_mul(r, a, m);
        a = mod_mul(a, a, m); b >>= 1;
    }
    return r;
}
ll mod_inv(ll a, ll m) {
    ll x, y, g = extended_gcd(a, m, x, y);
    if (g != 1 && g != -1) return -1;
    return (x % m + m) % m;
}
// Caution: m > 0; inverse exists iff gcd(a,m)
= 1.

// 4. Primality Test: O(sqrt(n))
bool is_prime(ll n) {
    if (n < 2) return false;
    if (n <= 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    for (ll i = 5; i <= n / i; i += 6)
        if (n % i == 0 || n % (i + 2) == 0)
return false;
    return true;
}

// 5. Sieve of Eratosthenes
void sieve() {
    vector<bool> prime(N, true);
    prime[0] = prime[1] = false;
    for (int i = 2; i <= (N - 1) / i; i++)
        if (prime[i])
            for (int j = i * i; j < N; j += i)
prime[j] = false;
    for (int i = 2; i < N; i++)
        if (prime[i]) primes.push_back(i);
}
// Caution: generates primes < N.

// 6. Smallest Prime Factor (SPF)
void compute_spf() {
    iota(spf, spf + N, 0);
    for (int i = 2; i <= (N - 1) / i; i++)
        if (spf[i] == i)
            for (int j = i * i; j < N; j += i)
                if (spf[j] == j) spf[j] = i;
}
map<ll, int> factor_spf(int n) {
    map<ll, int> res;
    while (n > 1) { res[spf[n]]++; n /= spf[n];
}
    return res;
}
// Caution: call compute_spf() first; requires
1 <= n < N.

// 7. Trial Factorization: O(sqrt(n)), no
preprocessing
map<ll, int> factorize(ll n) {
```

NUM (cont.)

```
    map<ll, int> res;
    for (ll p = 2; p <= n / p; p += (p == 2 ? 1
: 2))
        while (n % p == 0) { res[p]++; n /= p;
}
    if (n > 1) res[n]++;
    return res;
}

// 8. Factorization using precomputed primes
map<ll, int> factor_large(ll n) {
    map<ll, int> res;
    for (ll p : primes) {
        if (p > n / p) break;
        while (n % p == 0) { res[p]++; n /= p;
}
    }
    if (n > 1) res[n]++;
    return res;
}
// Caution: sieve must cover primes up to
sqrt(original n).

// 9. Divisors, Count and Sum
vector<ll> getDivisors(ll n) {
    vector<ll> d;
    for (ll i = 1; i <= n / i; i++)
        if (n % i == 0) {
            d.push_back(i);
            if (i != n / i) d.push_back(n / i);
        }
    sort(d.begin(), d.end());
    return d;
}
ll count_div(ll n) {
    ll ans = 1;
    for (auto [p, c] : factorize(n)) ans *= c +
1;
    return ans;
}
ll sum_div(ll n) {
    ll ans = 1;
    for (auto [p, c] : factorize(n)) {
        ll term = 1, cur = 1;
        for (int i = 0; i < c; i++) { cur *= p;
term += cur; }
        ans *= term;
    }
    return ans;
}
// Caution: n > 0; count/sum may overflow ll.

// 10. Distinct Prime Divisors for Every Number
void precompute_prime_divisors() {
    for (int p = 2; p < MAXP; p++)
        if (prime_divisors[p].empty())
            for (int j = p; j < MAXP; j += p)
                prime_divisors[j].push_back(p);
}
// Caution: call once; prime_divisors[x] stores
distinct prime factors.

// 11. Euler Totient: phi(n)
ll phi(ll n) {
```

NUM (cont.)

```
    ll ans = n;
    for (ll p = 2; p ≤ n / p; p++)
        if (n % p == 0) {
            while (n % p == 0) n /= p;
            ans -= ans / p;
        }
    if (n > 1) ans -= ans / n;
    return ans;
}

// 12. Totient Sieve
void phi_sieve() {
    iota(phi_arr, phi_arr + N, 0LL);
    for (int p = 2; p < N; p++)
        if (phi_arr[p] == p)
            for (int j = p; j < N; j += p)
                phi_arr[j] -= phi_arr[j] / p;
}

// 13. Mobius Function Sieve: requires SPF
void mobius_sieve() {
    fill(mobius, mobius + N, 1);
    mobius[0] = 0;
    for (int p = 2; p < N; p++)
        if (spf[p] == p) {
            for (int j = p; j < N; j += p)
mobius[j] *= -1;
            for (ll j = 1LL * p * p; j < N; j
+= 1LL * p * p)
                mobius[j] = 0;
        }
}
// Caution: call compute_spf() first.

// 14. Binary GCD: non-negative inputs
ll binary_gcd(ll a, ll b) {
    if (!a) return b;
    if (!b) return a;
    int shift = __builtin_ctzll((ull)(a | b));
    a >>= __builtin_ctzll((ull)a);
    do {
        b >>= __builtin_ctzll((ull)b);
        if (a > b) swap(a, b);
        b -= a;
    } while (b);
    return a << shift;
}

// 15. Legendre's Formula: exponent of prime p
in n!
ll power_in_fact(ll n, ll p) {
    ll cnt = 0;
    while (n) { n /= p; cnt += n; }
    return cnt;
}
// Caution: p must be prime.

// 16. Chinese Remainder Theorem
// x = a1 (mod m1), x = a2 (mod m2); returns -1
if inconsistent.
ll crt(ll a1, ll m1, ll a2, ll m2) {
    if (m1 ≤ 0 || m2 ≤ 0) return -1;
    ll x, y, g = extended_gcd(m1, m2, x, y);
    if ((i128)(a2 - a1) % g ≠ 0) return -1;
```

NUM (cont.)

```
    ll m2g = m2 / g, l = m1 * m2g;
    ll t = (ll)((i128)(a2 - a1) / g) * x % m2g);
    ll k = (t % m2g + m2g) % m2g;
    return (ll)(((i128)a1 + (i128)m1 * k) % l + l) % l;
}
// Caution: lcm and result must fit in ll.

// 17. Miller-Rabin: deterministic for unsigned 64-bit
integers
ull mulmod(ull a, ull b, ull m) { return (ull)((u128)a *
b % m); }
ull binpow(ull a, ull d, ull m) {
    ull r = 1;
    while (d) {
        if (d & 1) r = mulmod(r, a, m);
        a = mulmod(a, a, m); d >>= 1;
    }
    return r;
}
bool miller_rabin(ull n) {
    if (n < 2) return false;
    for (ull p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL,
17ULL, 19ULL, 23ULL})
        if (n % p == 0) return n == p;
    ull d = n - 1, s = 0;
    while (!(d & 1)) { d >>= 1; s++; }
    for (ull a : {2ULL, 325ULL, 9375ULL, 28178ULL,
450775ULL, 9780504ULL, 1795265022ULL}) {
        if (a % n == 0) continue;
        ull x = binpow(a % n, d, n);
        if (x == 1 || x == n - 1) continue;
        bool composite = true;
        for (ull r = 1; r < s; r++) {
            x = mulmod(x, x, n);
            if (x == n - 1) { composite = false; break;
}
        }
        if (composite) return false;
    }
    return true;
}

// 18. Pollard Rho: fast factorization for large
integers
mt19937_64
rng(chrono::steady_clock::now().time_since_epoch().count());
ull rho_f(ull x, ull c, ull mod) { return (mulmod(x, x,
mod) + c) % mod; }
ull pollard(ull n) {
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    while (true) {
        ull x = rng() % (n - 2) + 2, y = x, c = rng() %
(n - 1) + 1, d = 1;
        while (d == 1) {
            x = rho_f(x, c, n);
            y = rho_f(rho_f(y, c, n), c, n);
            ull diff = x > y ? x - y : y - x;
            d = gcd(diff, n);
        }
        if (d ≠ n) return d;
    }
}
void factor_rho(ull n, vector<ull>& factors) {
    if (n == 1) return;
    if (miller_rabin(n)) { factors.push_back(n); return;
}
    ull d = pollard(n);
    factor_rho(d, factors);
    factor_rho(n / d, factors);
}
// Caution: randomized; expected fast, but not a
guaranteed worst-case bound.

/* QUICK FORMULAS
gcd(a,b,c) = gcd(gcd(a,b),c)
lcm(a,b) = a / gcd(a,b) * b
Number of divisors = product(e_i + 1)
```

NUM (cont.)

```
Sum of divisors = product(1 + p + p^2 + ... +
p^e)
phi(n) = n * product(1 - 1/p), for distinct
prime p | n
Mobius: mu(1)=1; mu(n)=0 if a prime square
divides n
Legendre: exponent of p in n! =
floor(n/p)+floor(n/p^2)+...
CRT: compatible iff a1 = a2 (mod gcd(m1,m2))
*/

/* PRECOMPUTATION
sieve();
compute_spf();
precompute_prime_divisors();
phi_sieve();
mobius_sieve(); // requires compute_spf()
*/
```

MEX

```
#include <bits/stdc++.h>
using namespace std;

// 1. MEX using Frequency Array: O(n) time,
O(n) space
int mex_freq(const vector<int>& a) {
    int n = a.size();
    vector<bool> present(n + 1, false);
    for(int x : a)
        if(x >= 0 && x <= n) present[x] = true;

    int mex = 0;
    while(mex <= n && present[mex]) mex++;
    return mex;
}
// Best choice for most CP problems. Values > n
can be ignored.

// 2. MEX using Set: O(n log n) time, O(n)
space
int mex_set(const vector<int>& a) {
    set<int> st;
    for(int x : a)
        if(x >= 0) st.insert(x);

    int mex = 0;
    while(st.count(mex)) mex++;
    return mex;
}
// Easy to understand; set removes duplicates
automatically.

// 3. MEX using Sorting: O(n log n) time, O(n)
extra space
int mex_sort(vector<int> a) {
    sort(a.begin(), a.end());
    int mex = 0;
    for(int x : a) {
        if(x == mex) mex++;
        else if(x > mex) break;
```

MEX (cont.)

```
    }
    return mex;
}
// Passes by value, so the original array
remains unchanged.
// Negative values and duplicates are handled
automatically.
```

MATH

formula

```
/*
================ CP FORMULA NOTE
================

1) SUMMATION
Sum 1..n: n*(n+1)/2
Sum i^2: n*(n+1)*(2*n+1)/6
Sum i^3: [n*(n+1)/2]^2
Sum first n even: n*(n+1)
Sum first n odd: n^2
Sum k*(2*k-1): n*(n+1)*(4*n-1)/6

2) AP
Nth term: a+(n-1)*d
Sum: n*(2*a+(n-1)*d)/2
Number of terms: (last-first)/d+1

3) GP
Nth term: a*r^(n-1)
Sum (r≠1): a*(r^n-1)/(r-1)
Infinite sum (|r|<1): a/(1-r)

4) NUMBER THEORY
GCD: gcd(a,b)=gcd(b,a%b)
LCM: (a/gcd(a,b))*b
Prime check: O(sqrt(n))
Number of divisors: (a1+1)*(a2+1)*...
Sum of divisors: Π((p^(a+1)-1)/(p-1))
Euler Totient: phi(n)=n*Π(1-1/p)
Mod inverse (prime MOD): a^(MOD-2)%MOD
Power modulo: Binary exponentiation O(log b)

5) MODULAR ARITHMETIC
Addition: (a+b)%m
Subtraction: (a-b+m)%m
Multiplication: (a*b)%m
Division: a*modInverse(b)%m

6) COMBINATORICS
nCr: n!/(r!*(n-r)!)
Pascal: C(n,r)=C(n-1,r)+C(n-1,r-1)
Sum C(n,r): 2^n
Stars & Bars: C(n+k-1,k-1)

7) BIT MANIPULATION
Check bit i: (x>>i)&1
Set bit i: x|(1<<i)
Unset bit i: x&~(1<<i)
Toggle bit i: x^(1<<i)
```

formula (cont.)

```
Count bits: __builtin_popcount(x)

8) PREFIX SUM
pref[i]=a[0]+...+a[i-1]
Range sum [l,r]: pref[r+1]-pref[l]

9) TWO POINTERS / SLIDING WINDOW
Applications: Subarray sum, longest substring,
fixed/variable window

10) MATH TRICKS
XOR 1..n:
n%4==0 → n
n%4==1 → 1
n%4==2 → n+1
n%4==3 → 0
Sum of digits over 1..n: Digit DP

================================================
*/
int main(){
    return 0;
}/*
================ CP FORMULA NOTE
================

1) SUMMATION
Sum 1..n: n*(n+1)/2
Sum i^2: n*(n+1)*(2*n+1)/6
Sum i^3: [n*(n+1)/2]^2
Sum first n even: n*(n+1)
Sum first n odd: n^2
Sum k*(2*k-1): n*(n+1)*(4*n-1)/6

2) AP
Nth term: a+(n-1)*d
Sum: n*(2*a+(n-1)*d)/2
Number of terms: (last-first)/d+1

3) GP
Nth term: a*r^(n-1)
Sum (r≠1): a*(r^n-1)/(r-1)
Infinite sum (|r|<1): a/(1-r)

4) NUMBER THEORY
GCD: gcd(a,b)=gcd(b,a%b)
LCM: (a/gcd(a,b))*b
Prime check: O(sqrt(n))
Number of divisors: (a1+1)*(a2+1)*...
Sum of divisors: Π((p^(a+1)-1)/(p-1))
Euler Totient: phi(n)=n*Π(1-1/p)
Mod inverse (prime MOD): a^(MOD-2)%MOD
Power modulo: Binary exponentiation O(log b)

5) MODULAR ARITHMETIC
Addition: (a+b)%m
Subtraction: (a-b+m)%m
Multiplication: (a*b)%m
Division: a*modInverse(b)%m

6) COMBINATORICS
nCr: n!/(r!*(n-r)!)
Pascal: C(n,r)=C(n-1,r)+C(n-1,r-1)
Sum C(n,r): 2^n
```

formula (cont.)

```
Stars & Bars: C(n+k-1,k-1)

7) BIT MANIPULATION
Check bit i: (x>>i)&1
Set bit i: x|(1<<i)
Unset bit i: x&~(1<<i)
Toggle bit i: x^(1<<i)
Count bits: __builtin_popcount(x)

8) PREFIX SUM
pref[i]=a[0]+...+a[i-1]
Range sum [l,r]: pref[r+1]-pref[l]

9) TWO POINTERS / SLIDING WINDOW
Applications: Subarray sum, longest substring,
fixed/variable window

10) MATH TRICKS
XOR 1..n:
n%4==0 → n
n%4==1 → 1
n%4==2 → n+1
n%4==3 → 0
Sum of digits over 1..n: Digit DP

================================================
*/
int main(){
    return 0;
}
```

GRAPH

graph all

```
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define pii pair<int,int>
#define pll pair<ll,ll>

const int LOG=20;
const ll INF=1e18;

vector<vector<int>> adj_list;
vector<vector<pii>> weighted_adj_list;
vector<int> dist,parent,color;
vector<bool> visited,path_visited;
int n,m;

vector<vector<bool>> grid_visited;
vector<vector<int>> grid_dist;
vector<vector<pii>> grid_parent;
vector<pii> direction={{-1,0},{1,0},{0,-1},
{0,1}};

bool valid(int r,int c){return
r>=0&&r<n&&c>=0&&c<m;}

// 1. BFS
```

graph all (cont.)

```
void bfs(int src){
    queue<int> q;
    visited[src]=true;
    dist[src]=0;
    parent[src]=-1;
    q.push(src);
    while(!q.empty()){
        int u=q.front();q.pop();
        for(int v:adj_list[u]){
            if(!visited[v]){
                visited[v]=true;
                dist[v]=dist[u]+1;
                parent[v]=u;
                q.push(v);
            }
        }
    }
}

// 2. Shortest Path Reconstruction
vector<int> get_path(int dest){
    if(dist[dest]==-1)return {};
    vector<int> path;
    for(int
u=dest;u!=-1;u=parent[u])path.push_back(u);
    reverse(path.begin(),path.end());
    return path;
}

// 3. Multi-Source BFS
void multi_source_bfs(vector<int> sources){
    queue<int> q;
    for(int s:sources){
        if(!visited[s]){
            visited[s]=true;
            dist[s]=0;
            parent[s]=-1;
            q.push(s);
        }
    }
    while(!q.empty()){
        int u=q.front();q.pop();
        for(int v:adj_list[u]){
            if(!visited[v]){
                visited[v]=true;
                dist[v]=dist[u]+1;
                parent[v]=u;
                q.push(v);
            }
        }
    }
}

// 4. DFS
void dfs(int u){
    visited[u]=true;
    for(int v:adj_list[u])
        if(!visited[v])dfs(v);
}

// 5. Connected Components
int connected_components(int n){
    visited.assign(n,false);
    int cnt=0;
```

graph all (cont.)

```
    for(int i=0;i<n;i++){
        if(!visited[i]){
            cnt++;
            dfs(i);
        }
    }
    return cnt;
}

// 6. Grid DFS
void grid_dfs(int r,int c){
    grid_visited[r][c]=true;
    for(auto [dr,dc]:direction){
        int nr=r+dr,nc=c+dc;
        if(valid(nr,nc)&&!grid_visited[nr][nc])
{
            grid_parent[nr][nc]={r,c};
            grid_dfs(nr,nc);
        }
    }
}

// 7. Grid BFS
void grid_bfs(int sr,int sc){
    queue<pii> q;
    grid_visited[sr][sc]=true;
    grid_dist[sr][sc]=0;
    grid_parent[sr][sc]={-1,-1};
    q.push({sr,sc});
    while(!q.empty()){
        auto [r,c]=q.front();q.pop();
        for(auto [dr,dc]:direction){
            int nr=r+dr,nc=c+dc;
            if(valid(nr,nc)&&!grid_visited[nr]
[nc]){
                grid_visited[nr][nc]=true;
                grid_dist[nr][nc]=grid_dist[r]
[c]+1;
                grid_parent[nr][nc]={r,c};
                q.push({nr,nc});
            }
        }
    }
}

// 8. Grid Multi-Source BFS
void grid_multi_source_bfs(vector<pii> sources)
{
    queue<pii> q;
    for(auto [r,c]:sources){
        if(valid(r,c)&&!grid_visited[r][c]){
            grid_visited[r][c]=true;
            grid_dist[r][c]=0;
            grid_parent[r][c]={-1,-1};
            q.push({r,c});
        }
    }
    while(!q.empty()){
        auto [r,c]=q.front();q.pop();
        for(auto [dr,dc]:direction){
            int nr=r+dr,nc=c+dc;
            if(valid(nr,nc)&&!grid_visited[nr]
[nc]){
                grid_visited[nr][nc]=true;
```

graph all (cont.)

```
                grid_dist[nr][nc]=grid_dist[r]
[c]+1;
                grid_parent[nr][nc]={r,c};
                q.push({nr,nc});
            }
        }
    }
}

// 9. Undirected Cycle Detection (DFS)
bool cycle_undirected(int u,int p){
    visited[u]=true;
    for(int v:adj_list[u]){
        if(!visited[v]){
            if(cycle_undirected(v,u))return
true;
        }else if(v≠p)return true;
    }
    return false;
}

// 10. Directed Cycle Detection (DFS)
bool cycle_directed_dfs(int u){
    visited[u]=path_visited[u]=true;
    for(int v:adj_list[u]){
        if(path_visited[v])return true;

if(!visited[v]&&cycle_directed_dfs(v))return
true;
    }
    path_visited[u]=false;
    return false;
}

bool has_directed_cycle(int n){
    visited.assign(n,false);
    path_visited.assign(n,false);
    for(int i=0;i<n;i++)

if(!visited[i]&&cycle_directed_dfs(i))return
true;
    return false;
}

// 11. Bipartite Check (BFS)
bool bipartite(int n){
    color.assign(n,-1);
    for(int s=0;s<n;s++){
        if(color[s]≠-1)continue;
        queue<int> q;
        q.push(s);
        color[s]=0;
        while(!q.empty()){
            int u=q.front();q.pop();
            for(int v:adj_list[u]){
                if(color[v]==-1){
                    color[v]=color[u]^1;
                    q.push(v);
                }else
if(color[v]==color[u])return false;
            }
        }
    }
    return true;
```

graph all (cont.)

```
}


// 13. Dijkstra (Non-negative weights)
vector<ll> dijkstra(int src){
    int n=weighted_adj_list.size();
    vector<ll> d(n,INF);

priority_queue<pll,vector<pll>,greater<pll>>
pq;
    d[src]=0;
    pq.push({0,src});
    while(!pq.empty()){
        auto [du,u]=pq.top();pq.pop();
        if(du≠d[u])continue;
        for(auto [v,w]:weighted_adj_list[u]){
            if(d[v]>du+w){
                d[v]=du+w;
                pq.push({d[v],v});
            }
        }
    }
    return d;
}

// 14. 0-1 BFS (Weights 0 or 1)
vector<int> zero_one_bfs(int src){
    int n=weighted_adj_list.size();
    vector<int> d(n,1e9);
    deque<int> q;
    d[src]=0;
    q.push_front(src);
    while(!q.empty()){
        int u=q.front();q.pop_front();
        for(auto [v,w]:weighted_adj_list[u]){
            if(d[v]>d[u]+w){
                d[v]=d[u]+w;
                if(w==0)q.push_front(v);
                else q.push_back(v);
            }
        }
    }
    return d;
}

// 15. DSU / Union-Find
struct DSU{
    vector<int> p,sz;
    DSU(int n){
        p.resize(n);
        sz.assign(n,1);
        iota(p.begin(),p.end(),0);
    }
    int find(int u){
        return p[u]==u?u:p[u]=find(p[u]);
    }
    bool unite(int u,int v){
        u=find(u);v=find(v);
        if(u==v)return false;
        if(sz[u]<sz[v])swap(u,v);
        p[v]=u;
        sz[u]+=sz[v];
        return true;
    }
```

graph all (cont.)

```
    bool same(int u,int v){return
find(u)==find(v);}
    int size(int u){return sz[find(u)];}
};

struct Edge{
    int u,v,w;
};

// 16. Kruskal MST
ll kruskal(int n,vector<Edge> edges){
    sort(edges.begin(),edges.end(),
        [](Edge a,Edge b){return a.w<b.w;});
    DSU dsu(n);
    ll cost=0;
    int cnt=0;
    for(auto [u,v,w]:edges){
        if(dsu.unite(u,v)){
            cost+=w;
            cnt++;
        }
    }
    return cnt==n-1?cost:INF;
}

// 17. Prim MST
ll prim(int n){
    vector<bool> used(n,false);

priority_queue<pii,vector<pii>,greater<pii>>
pq;
    pq.push({0,0});
    ll cost=0;
    int cnt=0;
    while(!pq.empty()){
        auto [w,u]=pq.top();pq.pop();
        if(used[u])continue;
        used[u]=true;
        cost+=w;
        cnt++;
        for(auto [v,wt]:weighted_adj_list[u])
            if(!used[v])pq.push({wt,v});
    }
    return cnt==n?cost:INF;
}

// 18. Tree Diameter (Unweighted Tree)
pair<int,int> farthest_node(int src,int n){
    vector<int> d(n,-1);
    queue<int> q;
    q.push(src);
    d[src]=0;
    int far=src;
    while(!q.empty()){
        int u=q.front();q.pop();
        if(d[u]>d[far])far=u;
        for(int v:adj_list[u]){
            if(d[v]==-1){
                d[v]=d[u]+1;
                q.push(v);
            }
        }
    }
    return {far,d[far]};
```

graph all (cont.)

```
}

int tree_diameter(int n){
    auto [a,d1]=farthest_node(0,n);
    auto [b,d2]=farthest_node(a,n);
    return d2;
}
```

init

```
// Unweighted graph
cin>>n>>m;
adj_list.assign(n,{});
visited.assign(n,false);
dist.assign(n,-1);
parent.assign(n,-1);

for(int i=0;i<m;i++){
    int u,v;cin>>u>>v;
    adj_list[u].push_back(v);
    adj_list[v].push_back(u);
}

// Weighted graph
weighted_adj_list.assign(n,{});
for(int i=0;i<m;i++){
    int u,v,w;cin>>u>>v>>w;
    weighted_adj_list[u].push_back({v,w});
    weighted_adj_list[v].push_back({u,w});
}

// Grid
cin>>n>>m;
grid_visited.assign(n,vector<bool>(m,false));
grid_dist.assign(n,vector<int>(m,-1));
grid_parent.assign(n,vector<pii>(m,{-1,-1}));
```

lca

```
#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9, LG = 18;

vector<int> g[N];
int par[N][LG + 1], dep[N], sz[N];


// এই ফাংশনটির কাজ হলো ট্রির নোডগুলোর ডেপথ, সাবট্রি
সাইজ এবং বাইনারি লিফটিং টেবিল (par[u][i]) প্রাক-প্রসেস
করে তৈরি করে রাখা।
void dfs(int u, int p = 0)
{
    par[u][0] = p;
    dep[u] = dep[p] + 1;
    sz[u] = 1;
```

lca (cont.)

```
    for (int i = 1; i ⩽ LG; i++) par[u][i] =
par[par[u][i - 1]][i - 1]; //binary lifting
2^3⟶4+4
    for (auto v: g[u]) if (v ≠ p) {
        dfs(v, u);
        sz[u] += sz[v];
    }
}

// এই ফাংশনটির কাজ হলো নোড u এবং v-এর সর্বনিম্ন
সাধারণ পূর্বপুরুষ (Lowest Common Ancestor) বের করা।
int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    for (int k = LG; k ⩾ 0; k--) if
(dep[par[u][k]] ⩾ dep[v]) u = par[u][k];
    if (u == v) return u;
    for (int k = LG; k ⩾ 0; k--) if (par[u][k]
≠ par[v][k]) u = par[u][k], v = par[v][k];
    return par[u][0];
}



//এই ফাংশনটির কাজ হলো নোড u থেকে ঠিক k ধাপ উপরে
(তার Ancestors-এর দিকে) গেলে কোন নোডটি পাওয়া যাবে,
তা দ্রুত বের করা।
int kth(int u, int k) {
    assert(k ⩾ 0);
    for (int i = 0; i ⩽ LG; i++) if (k & (1 <<
i)) u = par[u][i];
    return u;
}



//এই ফাংশনটি গাছের যেকোনো দুটি নোড u এবং v-এর
মধ্যকার সর্বনিম্ন দূরত্ব (Path-এ মোট কতটি এজ বা লাইন
আছে) হিসাব করে।

int dist(int u, int v) {
    int l = lca(u, v);
    return dep[u] + dep[v] - (dep[l] << 1);
}


//kth node from u to v, 0th node is u
int go(int u, int v, int k) {
    int l = lca(u, v);

    int d = dep[u] + dep[v] - (dep[l] << 1);
    assert(k ⩽ d);
    if (dep[l] + k ⩽ dep[u]) return kth(u, k);
    k -= dep[u] - dep[l];
    return kth(v, dep[v] - dep[l] - k);
}

int32_t main() {
    int n; cin >> n;
    for (int i = 1; i < n; i++) {
        int u, v; cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
```

lca (cont.)

```
    dfs(1);
    int q; cin >> q;
    while (q--) {
        int u, v; cin >> u >> v;
        cout << dist(u, v) << '\n';
    }
    return 0;
}
```

DSU

DSU basic

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
using namespace std;
template <typename T>
using pbds = tree<T, null_type, less<T>,
rb_tree_tag,
tree_order_statistics_node_update>;

#define fast_io
ios_base::sync_with_stdio(false);
cin.tie(nullptr); cout.tie(nullptr)
#define ll long long
#define vi vector<int>
#define vll vector<ll>
#define pii pair<int, int>
#define pll pair<ll, ll>
#define pb push_back
#define all(v) (v).begin(), (v).end()
#define endl '\n'
#define forn(i, a, b) for(int i = a; i < b;
i++)
#define rev(i, a, b) for(int i = a; i ⩾ b; i
--)
const int MOD = 1e9+7;

/*basic DSU without OPTIMIZATION

const int MaxN=1e5+10;
int parent[MaxN];


void make_set(int v)
{
    parent[v]=v;
}

int find(int v)
{
    if(parent[v]=v) return v;
    return find(parent[v]);
}

void Union(int a, int b)
{
    int leader_a=find(a);
    int leader_b=find(b);
```

DSU basic (cont.)

```
    if(leader_a≠leader_b)
    {
        parent[leader_b]=leader_a;
    }

}


*/

//OPTIMIZED DSU Template


const int MaxN=1e5+10;
int parent[MaxN];
int sz[MaxN];

//create set with size
void make_set(int v)
{
    parent[v]=v;
    sz[v]=1;

}

int find(int v)
{
    if(parent[v]==v) return v;
    return parent[v]=find(parent[v]); //path
compression
}
//union by size
void Union(int a, int b)
{
    int leader_a=find(a);
    int leader_b=find(b);

    if(leader_a≠leader_b)
    {
        if(sz[leader_a]<sz[leader_b])
        {
            swap(leader_a,leader_b);
        }
        parent[leader_b]=leader_a;         //new
leader updated
        sz[leader_a]+=sz[leader_b];  // updated
size of new leader
    }

}
```

PBDS

orderset

```
#include<bits/stdc++.h>
#include<ext/pb_ds/assoc_container.hpp>
#include<ext/pb_ds/tree_policy.hpp>
using namespace std;
```

orderset (cont.)

```
using namespace __gnu_pbds;

#define ordered_set
tree<int,null_type,less<int>,rb_tree_tag,tree_order_statistics_node_upd
#define ordered_multiset
tree<pair<int,int>,null_type,less<pair<int,int>>,rb_tree_tag,tree_order

// PBDS Cheatsheet
// Time Complexity: O(log n) for insert, erase, find, bounds,
// find_by_order and order_of_key.

// ==================== ORDERED SET ====================

void ordered_set_operations(){
    ordered_set s;

    // Insert: O(log n)
    s.insert(10);
    s.insert(20);
    s.insert(30);
    s.insert(40);
    s.insert(20); // Duplicate ignored

    // Erase: O(log n)
    s.erase(20); // Erases value 20
    s.erase(100); // Does nothing if absent

    // Find: O(log n)
    if(s.find(30)≠s.end()) cout<<"Found\n";
    else cout<<"Not Found\n";

    // Size and empty: O(1)
    cout<<s.size()<<'\n';
    cout<<s.empty()<<'\n';

    // Lower bound: first element ≥ x
    auto it=s.lower_bound(25);
    if(it≠s.end()) cout<<*it<<'\n'; // 30

    // Upper bound: first element > x
    it=s.upper_bound(30);
    if(it≠s.end()) cout<<*it<<'\n'; // 40

    // K-th smallest: 0-indexed, O(log n)
    int k=1;
    if(k<(int)s.size())
        cout<<*s.find_by_order(k)<<'\n';

    // Count elements strictly smaller than x: O(log n)
    int x=30;
    cout<<s.order_of_key(x)<<'\n';

    // Count elements ≤ x
    int le=s.order_of_key(x)+(s.find(x)≠s.end());
    cout<<le<<'\n';

    // Count elements > x
    cout<<(int)s.size()-le<<'\n';

    // K-th largest: k is 0-indexed
    k=0;
    if(k<(int)s.size())
        cout<<*s.find_by_order((int)s.size()-1-k)<<'\n';

    // Erase using iterator
    it=s.find(30);
    if(it≠s.end()) s.erase(it);
}

// ==================== ORDERED MULTISET ====================
// Stores duplicates using {value, unique_id}.

void ordered_multiset_operations(){
    ordered_multiset ms;
    int id=0;

    // Insert duplicates
    ms.insert({10,id++});
    ms.insert({10,id++});
    ms.insert({20,id++});
    ms.insert({30,id++});

    // Size
    cout<<ms.size()<<'\n'; // 4
```

orderset (cont.)

```
    // Count elements strictly smaller than x
    int x=20;
    cout<<ms.order_of_key({x,-1})<<'\n'; // 2

    // Count elements ⩽ x
    int le=ms.order_of_key({x,INT_MAX});
    cout<<le<<'\n';

    // K-th smallest value: 0-indexed
    int k=1;
    if(k<(int)ms.size())
        cout<<ms.find_by_order(k)->first<<'\n';

    // Count occurrences of x
    int cnt=ms.order_of_key({x,INT_MAX})
           -ms.order_of_key({x,-1});
    cout<<cnt<<'\n';

    // Erase ONE occurrence of x
    auto it=ms.lower_bound({x,-1});
    if(it!=ms.end() && it->first==x)
        ms.erase(it);

    // Erase ALL occurrences of x
    while(true){
        auto it=ms.lower_bound({x,-1});
        if(it==ms.end() || it->first!=x) break;
        ms.erase(it);
    }
}

// ==================== QUICK REFERENCE ====================
//
// ordered_set s;
//
// s.insert(x);               // Insert x
// s.erase(x);                // Erase x
// s.find(x);                 // Find x
// s.lower_bound(x);          // First element ⩾ x
// s.upper_bound(x);          // First element > x
// *s.find_by_order(k);       // K-th smallest, 0-indexed
// s.order_of_key(x);         // Count elements < x
// s.size();                  // Number of elements
// s.empty();                 // Is empty?
//
// K-th largest (0-indexed):
// *s.find_by_order(s.size()-1-k);
//
// Count elements ⩽ x (integer set, overflow-safe):
// s.order_of_key(x)+(s.find(x)!=s.end());
//
// Count elements > x:
// s.size()-count_elements_le_x;
//
// ORDERED MULTISET:
// ms.insert({x,id++});       // Insert one occurrence
// ms.order_of_key({x,-1});   // Count values < x
// ms.find_by_order(k)->first;// K-th smallest value
//
// Notes:
// 1. find_by_order(k) is 0-indexed.
// 2. Check k < s.size() before dereferencing.
// 3. ordered_set does not allow duplicates.
// 4. Use unique IDs for ordered_multiset.
// 5. PBDS is a GNU C++ extension, not standard C++.
// 6. Typical operation complexity: O(log n).

int32_t main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // Use the functions above as needed.
    return 0;
}
```

DP

basic dp

```
#include<bits/stdc++.h>
using namespace std;

const int N=1005,INF=1e9;
int dp[N][N];

// 0/1 Knapsack: Each item can be taken at most once
int knapsack(int wt[],int val[],int w,int n){
    if(n==0 || w==0) return 0;

    int &ans=dp[n][w];
    if(ans!=-1) return ans;

    if(wt[n-1]⩽w)
        ans=max(val[n-1]+knapsack(wt,val,w-wt[n-1],n-1),
                knapsack(wt,val,w,n-1));
    else
        ans=knapsack(wt,val,w,n-1);

    return ans;
}

// Unbounded Knapsack: Each item can be taken unlimited times
int unbounded_knapsack(int wt[],int val[],int w,int n){
    if(n==0 || w==0) return 0;

    int &ans=dp[n][w];
    if(ans!=-1) return ans;

    if(wt[n-1]⩽w)
        ans=max(val[n-1]+unbounded_knapsack(wt,val,w-wt[n-1],n),
                unbounded_knapsack(wt,val,w,n-1));
    else
        ans=unbounded_knapsack(wt,val,w,n-1);
```

basic dp (cont.)

```
    return ans;
}

// LCS: Longest Common Subsequence
int lcs(string &a,string &b,int m,int n){
    if(m==0 || n==0) return 0;

    int &ans=dp[m][n];
    if(ans!=-1) return ans;

    if(a[m-1]==b[n-1])
        ans=1+lcs(a,b,m-1,n-1);
    else
        ans=max(lcs(a,b,m-1,n),lcs(a,b,m,n-1));

    return ans;
}

// Fibonacci
int fib(int n){
    if(n<=1) return n;

    int &ans=dp[n][0];
    if(ans!=-1) return ans;

    return ans=fib(n-1)+fib(n-2);
}

// Coin Change: Minimum coins, unlimited coins
int coin_change(int coin[],int n,int amount){
    if(amount==0) return 0;
    if(n==0) return INF;

    int &ans=dp[n][amount];
    if(ans!=-1) return ans;

    if(coin[n-1]<=amount)
        ans=min(1+coin_change(coin,n,amount-
coin[n-1]),
                coin_change(coin,n-1,amount));
    else
        ans=coin_change(coin,n-1,amount);

    return ans;
}

int32_t main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // Example: 0/1 Knapsack
    int n=3,w=5;
    int wt[]={1,2,3};
    int val[]={6,10,12};

    memset(dp,-1,sizeof(dp));
    cout<<knapsack(wt,val,w,n)<<'\n';

    return 0;
}
```

2 P

2p

```
// 1. Two Sum: Pair with target in sorted array
bool twoSum(vector<int>& a,int target){
    int l=0,r=(int)a.size()-1;
    while(l<r){
        int sum=a[l]+a[r];
        if(sum==target) return true;
        if(sum<target) l++;
        else r--;
    }
    return false;
}

// 2. Count pairs with sum <= K (sorted array)
long long countPairsLE(vector<int>& a,long long
k){
    sort(a.begin(),a.end());
    int l=0,r=(int)a.size()-1;
    long long cnt=0;
    while(l<r){
        if(a[l]+1LL*a[r]<=k){
            cnt+=r-l;
            l++;
        }
        else r--;
    }
    return cnt;
}

// 3. Count pairs with sum < K (sorted array)
long long countPairsLT(vector<int>& a,long long
k){
    sort(a.begin(),a.end());
    int l=0,r=(int)a.size()-1;
    long long cnt=0;
    while(l<r){
        if(a[l]+1LL*a[r]<k){
            cnt+=r-l;
            l++;
        }
        else r--;
    }
    return cnt;
}

// 4. Minimum difference between two elements
int minDifference(vector<int>& a){
    sort(a.begin(),a.end());
    int ans=INT_MAX;
    for(int i=1;i<(int)a.size();i++)
        ans=min(ans,a[i]-a[i-1]);
    return ans;
}

// 5. Merge two sorted arrays
vector<int> mergeSorted(vector<int>&
a,vector<int>& b){
    vector<int> c;
    int i=0,j=0;
    while(i<(int)a.size() && j<(int)b.size()){
        if(a[i]<=b[j]) c.push_back(a[i++]);
        else c.push_back(b[j++]);
    }
```

2p (cont.)

```
    while(i<(int)a.size()) c.push_back(a[i++]);
    while(j<(int)b.size()) c.push_back(b[j++]);
    return c;
}

// 6. Intersection of two sorted arrays (unique
values)
vector<int> intersection(vector<int>&
a,vector<int>& b){
    vector<int> ans;
    int i=0,j=0;
    while(i<(int)a.size() && j<(int)b.size()){
        if(a[i]==b[j]){
            if(ans.empty() || ans.back()!=a[i])
                ans.push_back(a[i]);
            i++,j++;
        }
        else if(a[i]<b[j]) i++;
        else j++;
    }
    return ans;
}

// 7. Longest subarray with sum ≤ K
// Requires non-negative array elements
int longestSumLEK(vector<int>& a,long long k){
    int l=0,ans=0;
    long long sum=0;
    for(int r=0;r<(int)a.size();r++){
        sum+=a[r];
        while(l<=r && sum>k) sum-=a[l++];
        ans=max(ans,r-l+1);
    }
    return ans;
}

// 8. Count subarrays with sum ≤ K
// Requires non-negative array elements
long long countSubarraysLEK(vector<int>& a,long
long k){
    if(k<0) return 0;
    int l=0;
    long long sum=0,cnt=0;
    for(int r=0;r<(int)a.size();r++){
        sum+=a[r];
        while(l<=r && sum>k) sum-=a[l++];
        cnt+=r-l+1;
    }
    return cnt;
}

// 9. Remove a target value in-place
// Returns new length; valid elements occupy
the prefix
int removeElement(vector<int>& a,int target){
    int slow=0;
    for(int fast=0;fast<(int)a.size();fast++){
        if(a[fast]!=target) a[slow++]=a[fast];
    }
    return slow;
}

// 10. Check if a sorted array contains a pair
with difference K
```

2p (cont.)

```
bool hasPairDifference(vector<int>& a,int k){
    sort(a.begin(),a.end());
    int l=0,r=1,n=a.size();
    while(r<n){
        if(l==r) r++;
        else if(a[r]-1LL*a[l]==k) return true;
        else if(a[r]-1LL*a[l]<k) r++;
        else l++;
    }
    return false;
}
```